

Butler 모델티어 B단계

실측벤치 수정개발지시서 v2.8

증거 무결성 · 안전 실행 · 재현 가능한 M3 실측

문서 상태	CONTROLLED_DRAFT_READY_FOR_IMPLEMENTATION
실행 상태	BLOCK_UNTIL_FULL_SHA_AND_POLICY_VALUES_RESOLVED
작성일	2026-07-14 (Asia/Seoul)
발신	보완팀5(온디바이스)
수신	총괄기획팀 · 알고리즘개발팀 · 메인개발팀 · 검증 · 실측 담당

본 문서는 v2.4의 방향을 유지하면서 구현 불가능하거나 거짓 합격을 만들 수 있는 계약을 교정한 실행 정보입니다. 확인되지 않은 저장소 상태·코드·성능·실측 결과는 완료로 취급하지 않습니다.

문서 구성

- 01 0. 최종 판정과 즉시 적용 사항
- 02 1. 문서 적용 범위와 규범
- 03 2. 목표 아키텍처와 신뢰 경계
- 04 3. 소스·Git·패키징 정보
- 05 4. P0-1 VolatileDraft 안전 계약
- 06 5. P0-2 단일 authoritative terminal gate
- 07 6. P0-3 모델 신원과 파일 교체 방지
- 08 7. P0-4 worker 프로토콜과 시간
- 09 8. P0-5 메모리·Metal·열 상태 계측
- 10 9. P0-6 dual physical resident
- 11 10. P0-7 독립 검증기 이중 구조
- 12 11. 벤치마크 정책과 통계
- 13 12. 보안·개인정보·egress
- 14 13. Evidence schema와 digest 체인
- 15 14. 실제 제품 entryptpoint와 코드 구조
- 16 15. 테스트 정보
- 17 16. CI 12-job과 의존관계
- 18 17. M3 실행 런북
- 19 18. 역할과 거버넌스
- 20 19. 산출물과 파일 구조
- 21 20. 최종 승인 기준
- 22 21. 금지 사항
- 23 22. 구현 우선순위
- 24 23. 2026년 기준 근거
- 25 부록 A. 보완팀5 종결 판정

0. 최종 판정과 즉시 적용 사항

0.1 현재 판정

```
SPEC_REVIEW_PASS=YES
CODE_IMPLEMENTATION_PASS=NO
BASE_FULL_SHA_RESOLVED=NO
BENCHMARK_POLICY_COMPLETE=NO
TRUST_POLICY_CONFIGURED=NO
CI_GREEN=NO
TARGET_M3_PREFLIGHT_PASS=NO
M3_START_ALLOWED=NO
ALL_46_DEFECTS_CLOSED=NO
M3_EVIDENCE_PASS=NO
INDEPENDENT_VERIFY_PASS=NO
G1_READY=NO
RUNTIME_ACTIVATION_ALLOWED=0
```

첨부 v2.7에서 확인된 사실은 3개 후보 패키지를 각각 45·54·42점으로 판정했고, P0 28건과 P1 18건(총 46건), 공격시험 11건 중 우회 재현 10건, M3 실측 0건을 기록했다는 점이다. 그러나 실제 저장소·전체 Git OID·CI 원본·서명 trust root·M3 raw는 첨부되지 않았다. 따라서 코드 구현, CI 통과, 실측 성능은 독립 확인하지 못했으며 현재 상태는 SPEC_READY뿐이다.

v2.7 본문은 “P0 28건 폐쇄 후 M3”라고 했지만, 동봉 결함표는 P1 18건까지 모두 M3_START 차단으로 지정했다. v2.8은 더 엄격한 기계 판독 자료를 정본으로 채택한다. **46건 전부가 폐쇄되고 각 행의 acceptance·attack evidence가 연결되기 전에는 M3를 시작하지 않는다.** P0는 merge와 M3 모두 비면제 차단, P1은 최소한 M3 readiness 비면제 차단이다.

0.2 v2.7에서 즉시 교정하는 14개 항목

번호	v2.7 표현	v2.8 강제 교정
C-01	d19badcd를 고정 SHA로 사용	전체 길이 commit OID와 tree OID를 각각 기록; 짧은 SHA만 있으면 BLOCK
C-02	제출 로그가 재실행 시 byte-동일	원시 로그는 시각·PID 때문에 달라질 수 있음; 입력이 같을 때 verdict.normalized.json만 byte-동일 강제
C-03	로그를 redact한 뒤 원문 0	안전 로그를 처음부터 생성; 민감정보 탐지 시 원본을 격리하고 실행을 재생성. 사후 가림으로 PASS 금지
C-04	raw draft zeroize	CPython 문자열의 완전 삭제를 주장하지 않음; bytearray 기반 최선 노력 덮어쓰기 + 수명 제한 + 전용 프로세스 종료
C-05	단일 모델 파일 SHA	단일 파일과 샷드·디렉터리 모델을 모두 지원하는 폐쇄형 Merkle 매니페스트 사용
C-06	Git blob SHA를 일반 SHA와 혼용	Git object format을 기록하고 git hash-object로 blob OID 비교; 배포 파일은 별도 SHA-256 매니페스트 사용
C-07	독립 verifier 하나가 모든 의미 재계산	실행 중 plaintext를 보는 live verifier와 사후

번호	v2.7 표현	v2.8 강제 교정
		digest·구조를 보는 artifact verifier의 책임을 분리
C-08	5 epoch AB/BA	각 epoch 안에서 반복 1은 AB, 반복 2는 BA로 균형; candidate당 fixture별 총 10회 측정
C-09	모든 RSS가 0보다 크면 통과	측정원·단위·대상 PID 집합·샘플 간격·누락률 ·시계까지 고정하고 정책 한도를 적용
C-10	P0가 달하면 성능 결론 가능	제품 SLO와 품질 하한이 사전 등록되지 않으면 실측은 가능해도 모델티어 판정은 NO_DECISION
C-11	signed attestation 존재 여부	서명 bundle만 보지 않고 trust root, issuer, identity, repository, workflow, ref, commit SHA를 모두 검증
C-12	worker nonce와 self-hash	nonce는 freshness만 증명한다. 승인 executable·부모 관측 stream·model/runtime digest·process tree를 함께 결속
C-13	모든 파일 byte scan, NUL 즉시 BLOCK	미등록 binary/NUL은 BLOCK. 등록 binary는 magic·MIME·digest·크기·archive 한도와 binary-aware secret scan을 모두 통과
C-14	결과 ZIP 자체 signed	inner immutable payload를 서명하고 bundle을 outer handoff ZIP에 동봉. ZIP이 자기 자신을 서명했다는 순환 주장은 금지

1. 문서 적용 범위와 규범

1.1 적용 범위

이 지시서는 다음 다섯 영역을 동시에 담는다.

- Butler 1.7B·4B 후보의 실제 제품 경로 비교 벤치마크
- soft repair, DLP, grounding, approval, policy의 종단 판정 일관성
- 모델 파일·런타임·소스·하네스·관측 패치의 재현 가능한 신원
- Apple Silicon M3 Max 목표 장비의 시간·메모리·열·오프라인 실행 증거
- 코드, CI, M3 실측, 독립 검증, M4 통합의 역할 및 완료 주장 경계

1.2 규범 용어

- **MUST/필수**: 미충족 시 해당 게이트를 BLOCK한다.
- **MUST NOT/금지**: 발견 즉시 해당 라운드를 무효화한다.
- **SHOULD/권고**: 미적용 사유와 대체 통제를 receipt에 기록한다.
- **MAY/선택**: 적용 여부가 합격 판정을 바꾸지 않는다.
- **UNSET**: 값이 아직 승인되지 않은 상태이며, 기본값으로 자동 대체하지 않고 BLOCK한다.

1.3 완료 주장 분리

상태	의미	금지되는 주장
SPEC_READY	지시서·스키마·테스트 계약이 완성됨	코드 완료, 실행 완료
CODE_READY_FOR_AUDIT	코드·테스트가 커밋되고 CI 후보가 생성됨	M3 적합, 성능 우수
READY_FOR_OWNER_M3_RUN	11종 시작 게이트가 모두 검증됨	M3 수치가 이미 유효함
M3_EVIDENCE_VALID_M4_PENDING	M3 raw와 독립 검증이 통과함	M4/G1 운영 준비 완료
G1_READY	별도 M4와 통합 게이트까지 통과함	이 문서만으로는 선언 불가

2. 목표 아키텍처와 신뢰 경계

2.1 실행 경계

```
Operator CLI
├─ Preflight Controller (Git·환경·정책·egress)
├─ Candidate Worker A (검증된 모델 + 실제 제품 파이프라인)
├─ Candidate Worker B (검증된 모델 + 실제 제품 파이프라인)
├─ Live Semantic Verifier (ephemeral plaintext scope)
├─ Evidence Writer (content-addressed safe receipts)
└─ Artifact Verifier (offline, producer 모듈 비의존)
```

2.2 raw 데이터 경계

1. prompt, reference, raw model output, repair draft는 후보 worker의 전용 프로세스 안에서만 plaintext로 존재한다.
2. quality 평가, grounding, DLP, approval/policy 판정은 같은 보호 경계 안에서 완료한다.
3. 부모 프로세스로는 승인된 최종 결과 또는 정책이 허용한 비식별 결과만 전달한다.
4. evidence에는 fixture ID, JCS SHA-256, 점수, 카운터, 시각, 모델 신원만 기록하고 원문·절대경로를 기록하지 않는다.
5. 독립 의미 검증이 원문을 필요로 하면 live verifier가 worker 경계 안에서 실행하고, 검증 receipt만 외부로 전달한다.
6. 사후 artifact verifier는 plaintext 의미를 다시 평가했다고 주장하지 않는다. 구조, digest 결속, 일정, 카운터, 환경, 정책을 재계산한다.

2.3 실패 폐쇄 원칙

누락, 중복, 순서 오류, 알 수 없는 필드, 0 또는 음수 측정값, digest 불일치, 시계 역행, sampler 오류, 정책값 UNSET, 외부 증거 부재는 예외적으로 통과시키지 않는다. UNKNOWN은 PASS가 아니다.

3. 소스·Git·패키징 정보

3.1 SHA/OID 필드

다음 필드는 모두 전체 길이 16진 객체 ID 또는 SHA-256이어야 한다.

필드	의미	생성 근거
BASE_SUBJECT_COMMIT_OID	개발 시작 commit	<code>git rev-parse <ref>^{commit}</code>
BASE_SUBJECT_TREE_OID	기준 소스 tree	<code>git rev-parse <commit>^{tree}</code>
EFFECTIVE_PRODUCT_COMMIT_OID	관측 코드 포함 실제 제품 commit	<code>git rev-parse HEAD^{commit}</code>
EFFECTIVE_PRODUCT_TREE_OID	실제 실행 소스 tree	<code>git rev-parse HEAD^{tree}</code>
HARNESS_COMMIT_OID	하네스 commit	하네스가 별도 저장소면 별도 OID
HARNESS_TREE_OID	하네스 tree	<code>git rev-parse <harness>^{tree}</code>
INSTRUMENTATION_PATCH_SHA256	승인된 관측 패치의 canonical binary diff	경로·base·effective를 고정한 diff
MODEL_MANIFEST_SHA256	모델 파일 집합의 JCS 매니페스트	§ 6
RUNTIME_CONFIG_SHA256	실행 설정의 JCS payload	§ 7
BENCHMARK_POLICY_SHA256	사전 등록 SLO·품질 정책	§ 11

`git rev-parse --show-object-format` 결과를 sha1 또는 sha256으로 기록한다. Git blob OID와 일반 파일 SHA-256을 같은 필드에 넣지 않는다.

3.2 기준 해소 게이트

```
set -euo pipefail
test -z "$(git status --porcelain=v1 --untracked-files=all)"
git fetch --prune origin
BASE_FULL="$(git rev-parse d19badcd^{commit})"
test "$(printf '%s' "$BASE_FULL" | wc -c | tr -d ' ')" -ge 40
git merge-base --is-ancestor "$BASE_FULL" origin/main
git fsck --full
git submodule status --recursive
git lfs ls-files --all 2>/dev/null || true
```

첨부물에 저장소가 없으므로 위 출력은 현재 문서에서 생성하지 않는다. 구현팀은 raw 출력에 경로·사용자명이 섞이지 않도록 안전 wrapper를 사용한다.

3.3 instrumentation digest

관측 패치는 별도 commit 또는 연속 commit range로 유지한다. digest 입력은 다음 정보를 포함한다.

```
base_commit_oid
effective_commit_oid
sorted_instrumented_paths
git_diff_binary_full_index_bytes
```

비관측 기능 변경이 같은 range에 섞이면 BLOCK한다. 관측 코드가 제품 의미를 바꾸지 않는다는 route-diff와 golden fixture 비교를 별도 실행한다.

3.4 재현 가능한 패키지

```
git archive --format=tar --prefix=product/ "$EFFECTIVE_PRODUCT_COMMIT_OID" \
| gzip -n > product-source.tar.gz
git archive --format=tar --prefix=harness/ "$HARNESS_COMMIT_OID" \
| gzip -n > harness-source.tar.gz
git ls-tree -r --full-tree "$EFFECTIVE_PRODUCT_COMMIT_OID" > product-git-tree.txt
git ls-tree -r --full-tree "$HARNESS_COMMIT_OID" > harness-git-tree.txt
git show --no-patch --format=fuller "$EFFECTIVE_PRODUCT_COMMIT_OID" > product-commit.txt
git show --no-patch --format=fuller "$HARNESS_COMMIT_OID" > harness-commit.txt
```

추가 강제사항:

- .gitattributes의 export-subst가 존재하면 archive bytes가 blob과 달라질 수 있으므로 금지하거나 별도 변환 매니페스트로 증명한다.
- export-ignore는 누락 경로 목록을 명시하고 하네스·스키마·테스트·라이선스가 빠지지 않게 한다.
- submodule commit, Git LFS object digest, 패키지 관리자 lockfile, Python/Swift/clang/Xcode/macOS build를 고정한다.
- 원시 실행 로그의 byte 동일성을 요구하지 않는다. 동일 입력에서 RFC 8785 정규화한 verdict.normalized.json과 schedule digest의 byte 동일성을 요구한다.
- Git LFS pointer blob과 checkout된 대용량 객체를 혼동하지 않는다. pointer OID→LFS object SHA-256 매핑, git lfs fsck, materialized file digest를 별도로 기록한다. git archive가 pointer를 내보내는 경우 materialized LFS payload는 승인 매니페스트로 별도 패키징한다.
- source blob 등가는 Git object 기준, 배포 file 등가는 SHA-256 기준으로 각각 전수 검증한다. submodule은 URL·full commit OID·tree OID·재귀 상태를 기록한다.
- 배포는 release.tar.gz를 먼저 결정론적으로 만든 뒤 이 payload digest를 in-toto Statement v1 subject로 삼고 detached Sigstore bundle을 생성한다. release.tar.gz, bundle, trust policy, 검증 명령을 outer handoff ZIP에 동봉한다. outer ZIP 자체의 SHA-256은 외부 전달 채널에 기록한다.

3.5 서명·provenance 검증 계약

서명 파일의 존재는 PASS가 아니다. 검증기는 trust_policy.json의 다음 값을 모두 강제하며 UNSET, wildcard identity, 무제한 issuer를 거부한다.

```
{
  "mode": "keyless",
  "certificate_oidc_issuer": "UNSET",
  "certificate_identity": "UNSET",
  "repository": "UNSET",
  "workflow": "UNSET",
  "ref": "UNSET",
  "commit_sha": "UNSET",
  "subject_name": "release.tar.gz",
  "subject_sha256": "UNSET"
}
```


keyless는 Sigstore bundle의 certificate·transparency log inclusion proof·identity claim을 검증한다. key/KMS 방식은 승인 public key 또는 KMS URI와 key version을 정책에 고정한다. 어떤 방식이든 attestation subject digest, source revision, policy digest, CI artifact digest가 현재 payload와 정확히 일치해야 한다. trust root를 패키지 제작자가 임의 교체할 수 없도록 조직 승인 채널에서 별도 고정한다.

4. P0-1 VolatileDraft 안전 계약

4.1 구현 규칙

- raw draft 보유 객체는 dataclass, Pydantic model, dict, tuple로 구현하지 않는다.
- 저장소는 bytearray 또는 동등한 mutable buffer를 사용한다. immutable str 복사본의 완전 삭제를 주장하지 않는다.
- repr과 str은 예외를 던지지 않고 <VolatileDraft REDACTED state=...>만 반환한다. 로거가 객체를 출력해도 raw가 나오지 않아야 한다.
- pickle, copy, deepcopy, JSON encoder, dataclass 변환은 TypeError로 차단한다.
- raw를 반환하는 consume() 대신 같은 프로세스의 승인된 callback으로 consume_into(callback)를 사용한다.
- request digest와 draft digest를 생성 시점 및 사용 직전에 constant-time 비교한다.
- 상태는 FRESH → BORROWED → DESTROYED만 허용한다. 두 번째 사용, 중첩 borrow, 예외 후 재사용은 BLOCK한다.
- callback 종료의 finally에서 buffer를 덮어쓰고, 고민감 요청은 worker 프로세스를 종료한다.
- hard/DLP/approval/identity/asset/policy 실패에서는 capture와 repair 카운터가 모두 0이어야 한다.

4.2 참조 코드 계약

```
class VolatileDraft:
    __slots__ = ("_buf", "_request_sha256", "_draft_sha256", "_state")

    def __repr__(self) -> str:
        return f"<VolatileDraft REDACTED state={self._state}>"

    __str__ = __repr__

    def __reduce_ex__(self, protocol):
        raise TypeError("VolatileDraft is not serializable")

    def __copy__(self):
        raise TypeError("VolatileDraft is not copyable")

    def __deepcopy__(self, memo):
        raise TypeError("VolatileDraft is not copyable")

    def consume_into(self, request_sha256: bytes, callback):
        if self._state != "FRESH":
            raise DraftStateError("exactly-once violation")
        if not hmac.compare_digest(request_sha256, self._request_sha256):
            raise DraftBindingError("request mismatch")
        self._state = "BORROWED"
        try:
            return callback(memoryview(self._buf).toreadonly())
        finally:
            self._buf[:] = b"\x00" * len(self._buf)
            self._state = "DESTROYED"
```

이 코드는 방향을 고정하는 참조이며, memoryview를 callback 밖에 보관하지 못하도록 타입·테스트·코드리뷰로 강제한다. CPython allocator나 런타임 내부 복사까지 제거했다고 주장하지 않는다.

5. P0-2 단일 authoritative terminal gate

5.1 상태와 불변식

```
START
→ IDENTITY_VALIDATED
→ GENERATION_DONE
→ PROVISIONAL_QUALITY_DECIDED
→ GROUNDING_DONE
→ DLP_DONE
→ APPROVAL_POLICY_DONE
→ TERMINALIZED
```

- `provisional_quality_gate`는 최종 승인값이 아니다.
- `final_gate`는 모든 필수 단계 이후 `_terminalize()`에서 정확히 1회 생성한다.
- 모든 early return과 예외는 `_terminalize(fail_class=...)`를 통과한다.
- `verdict.status`, `verdict.fail_class`, `final_gate`, observer terminal event, CLI exit code가 동일한 판정을 나타내야 한다.
- terminal 이후 이벤트, 중복 stage, stage 순서 역전, stale final, terminal override를 허용하지 않는다.
- DLP가 present인 것과 `passed=true`인 것을 구분한다. `grounding digest`의 존재만으로 성공 처리하지 않는다.

5.2 종료 분류

fail_class	예	repair 허용
IDENTITY_BLOCK	모델 매니페스트 불일치	0
ASSET_BLOCK	test asset, 미승인 파일	0
HARD_POLICY_BLOCK	권한·정책 위반	0
DLP_BLOCK	DLP 실패·미실행	0
GROUNDING_BLOCK	필수 근거 실패	정책에 따름, DLP 전 raw 경계 안에서만
QUALITY_SOFT_FAIL	승인된 soft repair 대상	최대 1회, 정책값에 따름
RUNTIME_ERROR	worker·sampler·OOM	0, round invalid

6. P0-3 모델 신원과 파일 교체 방지

6.1 모델 매니페스트

모델이 파일 하나라는 가정을 폐기한다. 모델 루트 아래 정규 파일을 정렬해 다음 leaf를 만든다.

```
{
  "relative_path": "weights/model-00001.safetensors",
  "size_bytes": 123,
  "mode": "100644",
  "sha256": "...
}
```

- 경로는 UTF-8 NFC, / 구분자, 상대경로만 허용한다.
- .., 절대경로, symlink, socket, device, sparse surprise를 금지한다. 기본은 `st_nlink == 1`이다. 조직이 승인한 content-addressed immutable store만 예외로 하며 store root·object digest·read-only mount evidence를 정책에 고정한다.
- leaf를 relative_path 순으로 정렬하고 RFC 8785 JCS bytes에 SHA-256을 적용한다.
- tokenizer, config, adapters, vocabulary, runtime-compiled artifact도 매니페스트에 포함한다.
- 모델 라이선스·출처·허용된 사용범위를 별도 metadata digest로 결속한다.

6.2 검증 시점

7. parent가 승인 registry에서 variant_id + model_manifest_sha256 + runtime_config_sha256를 찾는다.
8. worker가 디렉터리 descriptor 기준 `openat`과 `O_NOFOLLOW`로 각 경로를 열고 `fstat` 후 열린 descriptor에서 SHA-256을 계산한다.
9. loader는 검증된 파일 집합과 동일한 루트만 사용한다.
10. generation 직전 매니페스트를 재계산한다.
11. inference 종료 직후에도 identity를 재확인하고 receipt에 pre/post digest를 기록한다.

APFS의 inode/dev/mtime는 보조 증거이며 digest를 대신하지 않는다. 모델 디렉터리 교체, 파일 swap, symlink 삽입, 승인되지 않은 test asset, 빈 digest는 모두 BLOCK한다.

7. P0-4 worker 프로토콜과 시간

7.1 JSONL FSM

```
READY_PRELOAD → MODEL_LOADED → INFERENCE_START → FIRST_CHUNK
→ INFERENCE_END → RESULT → CLOSED → EOF + returncode 0
```

각 이벤트 필수 필드:

```
schema_version, run_id, worker_id, seq, event_type,
monotonic_ns, wall_time_utc, payload_sha256, safe_payload
```

- seq는 0부터 1씩 증가한다.
- monotonic_ns는 엄격히 증가한다. wall clock은 표시용이며 duration 계산에 사용하지 않는다.
- JSON 한 줄 최대 64 KiB, 이벤트 최대 128개, worker deadline은 policy에서 고정한다.
- stdout은 프로토콜 JSONL 전용이다. stderr도 raw prompt/output을 쓰지 않는 안전 구조 로그만 허용한다.
- newline 없는 마지막 stdout/stderr bytes가 1바이트라도 있으면 digest·byte count에 넣고, stdout partial은 BLOCK한다.
- RESULT와 CLOSED는 정확히 1개, CLOSED 뒤 이벤트는 0개, EOF와 returncode 0을 확인한다.
- timeout 시 새 process group을 TERM 후 제한시간 내 KILL하고 자손 프로세스 잔존을 검사한다.

parent는 예측 불가능한 256-bit nonce, expected executable digest, model/runtime digest, request digest를 worker에 전달하고 worker event 전체에 challenge digest를 결속한다. parent가 request pipe write 완료와 첫 실제 stdout byte를 별도 monotonic clock으로 관측해 worker 자체 시각을 교차검증한다. stdout/stderr는 동시에 bounded drain하며 stream별 최대 4 MiB를 넘으면 BLOCK한다.

중요한 한계: nonce와 self-hash만으로 악성 trusted worker가 실제 물리 추론을 했다는 사실까지 증명할 수 없다. v2.8은 승인·서명된 worker build provenance, parent가 관측한 process/executable identity, model file descriptor digest, 실제 output stream digest, tokenizer 재계산, 제품 observer trace를 함께 요구한다. 이는 소프트웨어 증거의 신뢰 경계를 명시하는 것이며 하드웨어 원격 attestation을 허위로 주장하지 않는다.

7.2 TTFT 정의

$TTFT = FIRST_CHUNK.monotonic_ns - INFERENCE_START.monotonic_ns$. FIRST_CHUNK는 1개 이상의 최종 tokenizer token에 해당하는 비어 있지 않은 출력이어야 한다. preload·tokenization 포함 여부는 benchmark_policy.json에서 고정하고 candidate 간 동일하게 적용한다.

8. P0-5 메모리·Metal·열 상태 계측

8.1 메모리 receipt

필수 mark:

```
preload_baseline
model_loaded
load_peak
inference_start
first_chunk
inference_peak
inference_end
after_close
```

각 mark는 다음을 가진다.

```
monotonic_ns, target_pid_set, rss_bytes_total, available_memory_bytes,
memory_pressure, sample_source, sample_interval_ms, missed_sample_count,
metal_current_allocated_size_bytes, metal_recommended_max_working_set_size_bytes
```

- RSS는 worker와 승인된 자손 PID의 합계로 정의한다.
- 모든 byte 값은 정수이며 0보다 커야 한다. Metal 필드가 API에서 지원되지 않으면 unsupported_reason을 기록하고, 사전 정책이 허용하지 않으면 BLOCK한다.
- sampler thread 오류, PID 누락, 샘플 간격 초과, missed sample 비율 초과, memory pressure UNKNOWN은 round invalid이다.
- 메모리 한도는 장비 RAM의 임의 비율로 코드에 하드코딩하지 않고 사전 승인 정책에서 읽는다.

8.2 환경 고정

- ProcessInfo.thermalState는 epoch pre/post에 기록한다. serious 또는 critical이면 epoch invalid이다.
- Low Power Mode는 false여야 한다.
- macOS build, M3 Max GPU/CPU core 수, RAM, 전원 연결 상태, 배터리 상태, active processor count, Metal device registry ID를 기록한다.
- MTLDevice.currentAllocatedSize와 recommendedMaxWorkingSetSize는 지원 범위에서 기록한다.
- background process allowlist와 금지 프로세스 탐지 결과를 receipt로 남긴다.

9. P0-6 dual physical resident

두 모델 객체를 같은 프로세스에 만들었다는 사실만으로 통과시키지 않는다.

필수 조건:

12. candidate A/B는 서로 다른 승인 `variant_id`와 서로 다른 `MODEL_MANIFEST_SHA256`을 가진다.
13. 별도 worker process와 PID를 사용한다.
14. 두 worker가 `MODEL_LOADED` 상태인 겹치는 구간이 정책의 `min_co_residency_seconds` 이상이다.
15. 겹치는 구간 안에서 각각 실제 제품 fixture 1건 이상을 추론한다.
16. inference 직전에 양쪽 모델 매니페스트를 worker-side로 재해시한다.
17. 각 worker의 baseline/loaded/peak/after-close RSS와 전체 시스템 available memory를 기록한다.
18. policy bypass count는 observer 원본 이벤트에서 재계산해 0이어야 한다.
19. OOM, jetsam 유사 종료, MemoryError, sampler error, swap 급증 정책 초과는 BLOCK한다.

10. P0-7 독립 검증기 이중 구조

10.1 Live Semantic Verifier

목적: raw가 파괴되기 전에 DLP·grounding·quality 의미를 독립 확인한다.

- producer의 boolean을 입력으로 받지 않는다.
- fixture ID로 승인 evaluator를 직접 호출한다.
- DLP·grounding·quality 규칙과 evaluator artifact digest를 기록한다.
- plaintext를 evidence로 내보내지 않는다.
- producer와 다른 entrypoint를 사용하고 동일 모듈의 helper를 그대로 import하지 않는다.

10.2 Offline Artifact Verifier

목적: 제출 패키지의 구조·결속·계산을 네트워크 없이 재현한다.

- artifact_manifest.js.json을 먼저 읽고 모든 파일 SHA-256을 확인한다.
- JSON Schema Draft 2020-12 closed-world 검증을 수행한다.
- 모든 receipt의 JCS SHA-256과 parent/subject digest를 다시 계산한다.
- worker FSM, schedule, warmup 제외, cooldown elapsed, AB/BA, thermal/LPM, 메모리, dual overlap을 원본 필드에서 재계산한다.
- OS egress와 CI receipt를 파일 이름이나 boolean이 아니라 subject digest와 정책값으로 검증한다.
- orphan, duplicate digest, unknown schema, 누락 파일, 매니페스트 밖 파일을 BLOCK한다.
- 결과 디렉터리, CI/test/verifier 로그 전체에 raw/path/secret scanner를 새로 실행한다.
- producer 패키지의 Python path를 import하지 않고, 별도 배포 artifact와 digest를 가진다.

10.3 stable exit code

code	의미
0	모든 필수 게이트 PASS
10	schema/closed-world 실패
11	digest/provenance 실패
12	protocol/schedule 실패
13	security/privacy 실패
14	environment/memory 실패
15	evidence missing/orphan
20	내부 verifier 오류; PASS로 변환 금지

11. 벤치마크 정책과 통계

11.1 사전 등록 필수값

benchmark_policy.json에 다음 값을 실측 전 승인한다. 하나라도 UNSET이면 M3 시작을 차단한다.

```
fixture_manifest_sha256
candidate_A_variant_id / candidate_B_variant_id
quality_floor_by_task
quality_noninferiority_margin_by_task
ttft_slo_ms_by_task
p95_e2e_slo_ms_by_task
peak_rss_limit_bytes
available_memory_floor_bytes
max_repair_rate / max_cascade_rate
worker_deadline_seconds
min_co_residency_seconds
sampler_interval_ms / max_missed_sample_ratio
tokenizer_manifest_sha256
bootstrap_seed / bootstrap_iterations
```

숫자는 이 문서가 임의로 만들지 않는다. 제품 SLO와 승인된 품질 기준에서 가져와야 한다.

승인 정보는 YAML이 아니라 RFC 8785로 정규화 가능한 JSON이다. YAML은 편집 입력으로만 허용하며 duplicate key, alias/anchor, custom tag, non-string key, NaN/Infinity, 암묵적 날짜형을 거부한 뒤 JSON Schema를 통과시켜야 한다. 승인자는 canonical JSON digest에 서명한다. 런타임은 원 YAML을 다시 읽지 않는다.

```
def load_approved_policy(path, schema, expected_sha256):
    obj = strict_json_load(path) # duplicate/NaN/unknown type reject
    validate_closed_world(schema, obj) # unevaluatedProperties=false
    reject_unset_or_wildcards(obj)
    canonical = rfc8785(obj)
    if sha256(canonical).hexdigest() != expected_sha256:
        raise GateBlocked("POLICY_DIGEST_MISMATCH")
    return FrozenPolicy(obj)
```

11.2 고정 실행 수

- 5 epoch
- candidate당, fixture당, epoch당 measured run 2회
- 따라서 candidate당 fixture별 measured run 총 10회
- epoch마다 candidate별 대표 warmup 2회, 총 warmup 4회
- warmup은 실제 제품 entrypoint·동일 모델·동일 runtime config를 사용하되 측정 통계에서 제외
- measured 반복 1은 AB, 반복 2는 BA로 실행해 각 epoch 안에서 물리 순서를 균형화
- epoch 사이 observed cooldown은 monotonic clock으로 15.0초 이상이며, 단순히 설정값만 기록하지 않는다.

11.3 schedule digest

행별 필드:

```
epoch, task_id, fixture_id, candidate_id, model_manifest_sha256,
runtime_config_sha256, phase(warmup|measured), repetition,
physical_position, deterministic_seed
```

행을 실행 순서대로 RFC 8785 JCS 배열로 만들고 SHA-256한다. candidate·순서·warmup·반복을 빼면 안 된다. 실행 중 schedule 변경은 허용하지 않으며, 중단 후 재개는 새 run_id로 처음부터 시작한다.

11.4 보고 지표

영역	필수 지표
반응성	TTFT median/p95, end-to-end median/p95
생성	output tokens/sec, inter-token latency p50/p95, token count
품질	task별 score, quality floor pass, paired difference
전력	검증된 계측원·단위·시계가 있을 때만 energy 및 joules/request

energy는 검증된 계측원과 구간 결속이 있을 때만 수치로 보고한다. 추정 전력×시간이나 임의 joule 환산을 금지한다. 정책에서 energy를 필수 판정측으로 승인했는데 계측원이 없으면 NO_DECISION/BLOCK; 선택 지표면 NOT_MEASURED로 기록한다.

복구	repair rate, cascade rate, false repair count
안전	DLP block/pass, grounding pass, policy bypass count, egress allowed count
자원	model load time, peak RSS, loaded delta, available memory floor, Metal allocation
안정성	crash, timeout, OOM, sampler error, invalid epoch count

token 수는 고정 tokenizer 매니페스트로 재계산한다. 후보별 tokenizer가 다르면 공통 평가 tokenizer 또는 두 수치를 함께 보고하고 비교 기준을 사전 등록한다.

11.5 통계 규칙

- raw 10개 측정점을 fixture·candidate별로 보존한다.
- paired fixture 차이를 기본 비교 단위로 사용한다.
- median, p95와 함께 고정 seed의 bootstrap 95% 신뢰구간을 보고한다.
- 결측·무효 run을 조용히 제외하지 않는다. 원인과 수를 표시하고 전체 epoch를 재실행한다.
- 여러 task를 하나의 평균으로 숨기지 않는다. task별 결과와 가중 종합을 함께 보고한다.
- 관측 패치 overhead는 instrumentation on/off A/B로 별도 측정하고, 정책 한도를 초과하면 벤치 결과를 무효화한다.
- 에너지 지표는 권한·측정원이 검증된 경우에만 joules/request와 tokens/joule을 보고한다. 측정하지 못하면 NOT_MEASURED로 표시하며 추정값을 만들지 않는다.

11.6 모델티어 판정

최종 상태는 다음 중 하나만 허용한다.

- PROMOTE_1_7B_FOR_TASK_SET
- PROMOTE_4B_FOR_TASK_SET
- ROUTE_BY_TASK_AND_MEMORY
- NO_DECISION_POLICY_INCOMPLETE
- NO_DECISION_EVIDENCE_INVALID

어떤 모델도 단일 M3 결과만으로 전 기기·전 업무에 부적합하다고 낙인찍지 않는다. 판정에는 task set, hardware profile, runtime config, 정책 digest를 함께 표시한다.

12. 보안·개인정보·egress

12.1 안전 로그 생성

- prompt/output/reference, 사용자명, home 경로, 절대 모델 경로, 토큰·키·쿠키, 환경변수 값은 로그 금지다.
- logger API는 raw string을 받지 않고 fixture ID, digest, enum, 정수만 받는 typed event builder를 사용한다.
- 예외 객체의 message/args/traceback에 raw가 들어갈 수 있으므로 외부 evidence에는 stable error code와 redacted module/line ID만 기록한다.
- scanner가 민감정보를 발견하면 해당 로그를 사후 가림해 PASS시키지 않는다. 원본을 격리하고 원인을 수정한 뒤 새 run_id로 재 실행한다.
- 배포용 사본에 가림이 필요하면 redaction_applied=true와 원본 invalid 상태를 기록한다. 이 사본은 raw-zero PASS 증거가 아니다.

12.2 OS egress receipt

필수 증거:

```
controller_type, controller_binary_sha256, policy_sha256,
activation_monotonic_ns, deactivation_monotonic_ns,
target_pid_set, denied_attempt_count, allowed_attempt_count,
native_ipv4_probe, native_ipv6_probe, dns_probe, https_probe,
subprocess_probe, receipt_subject_sha256
```

- OS 차단 장치가 실제 M3에서 활성화되지 않으면 M3를 시작하지 않는다.
- 앱 내부 network_allowed=false boolean만으로 통과하지 않는다.
- native socket과 subprocess를 모두 시험한다.
- 허용된 outbound attempt은 0이어야 한다. local IPC나 loopback 필요성은 정책에서 별도 구분한다.
- 승인 가능한 macOS 프로파일은 (a) 서명·entitlement가 검증된 Network Extension content filter, (b) 전용 benchmark user 와 독립 검증된 OS deny controller, (c) 외부 deny-all VLAN/firewall 중 하나다. 어떤 프로파일도 준비·실증되지 않으면 OS_EGRESS_RECEIPT_VALID=NO다.
- PF 등 도구가 임의 PID/process group을 직접 결속한다고 가정하지 않는다. 실제 controller의 적용 단위와 자손 포함 방식, 재부팅/권한 요구, 실패 시 기본동작을 시험으로 증명한다.
- controller 권한·구현 방식은 실제 운영 가능한 도구로 고정하고, 존재하지 않는 macOS 보안 기능을 문서상 가정하지 않는다.

13. Evidence schema와 digest 체인

13.1 공통 envelope

모든 JSON receipt는 JSON Schema Draft 2020-12와 `unevaluatedProperties:false`를 사용한다.

```
{
  "schema_version": "butler.model-tier-b.receipt.v2.8",
  "receipt_type": "cold_worker",
  "run_id": "uuid",
  "subject": [{"name": "...", "sha256": "..."}],
  "parents": [{"receipt_sha256": "..."}],
  "payload": {},
  "created_at_utc": "RFC3339",
  "canonicalization": "RFC8785-JCS",
  "receipt_sha256": "..."
}
```

digest 계산 시 `receipt_sha256` 필드는 제외한다. 숫자는 finite JSON number만 허용하며 NaN/Infinity를 금지한다. 시간 `duration`은 monotonic 정수 나노초로 계산한다.

13.2 필수 receipt

```
source_provenance
environment
benchmark_policy
model_identity_A / model_identity_B
runtime_config_A / runtime_config_B
fixture_manifest
schedule
worker_event_stream_index
cold_worker_per_run
live_semantic_verify_per_run
epoch
dual_resident
os_egress
ci
raw_scan
instrumentation_overhead
artifact_manifest
final_verdict
```

receipt가 파일로 존재한다는 이유로 통과시키지 않는다. subject/parent digest 그래프가 하나의 `final_verdict`로 연결되어야 하며 orphan은 0개여야 한다.

artifact profile은 최소 receipt type, cardinality, final pointer, schema digest를 서명 정책에 고정한다. receipt 0개, manifest-only, final pointer가 가리키지 않는 verdict, 중복 digest, cycle은 모두 BLOCK한다.

13.3 전 파일 · archive · binary 스캐너

20. manifest에 등록되지 않은 파일, 미등록 binary, NUL 포함 미등록 파일은 즉시 BLOCK한다.
21. 등록 binary는 상대경로, magic bytes, MIME, schema/profile, exact SHA-256, 최대 byte 수가 모두 일치해야 한다.
22. text와 binary 모두 ASCII/UTF-8/UTF-16LE/BE printable run, token/key/absolute-path 패턴을 검사한다. “binary라 서 skip”은 금지한다.
23. ZIP/TAR/GZIP 등 container는 재귀 검사한다. 기본 한도는 정책값이며 `max_depth`, `max_entries`, `max_total_expanded_bytes`, `max_single_file_bytes`, `max_compression_ratio`를 초과하면 BLOCK한다.

24. symlink, hardlink, special file, path traversal, Unicode confusable path, case-fold collision, duplicate archive entry는 BLOCK한다.
25. finding에는 민감 원문을 쓰지 않고 rule ID, file digest, byte range, encoding만 기록한다.

14. 실제 제품 entrypoint와 코드 구조

14.1 호출해야 할 제품 경로

- Box3ProductPipelineRuntime
- run_product_matrix
- run_cold_worker_handshake
- measure_dual_resident_pressure
- verify_samples_first

이름만 검색하는 static token 검사는 완료 증거가 아니다. integration test는 import path, call graph, fixture가 실제 production gate를 통과했음을 coverage와 observer event로 증명한다.

14.2 권장 모듈 구조

```
butler_bench/
  cli.py
  policy.py
  provenance.py
  model_manifest.py
  volatile_draft.py
  terminal_gate.py
  worker_protocol.py
  memory_sampler.py
  metal_probe.swift
  environment_probe.swift
  egress_controller.py
  live_verifier.py
  artifact_writer.py
  schedule.py
  statistics.py
  dual_resident.py
  schemas/
  verifier_offline/
tests/
  unit/
  contract/
  integration/
  mutation/
  adversarial/
scripts/
  canonical_preflight.sh
  package_from_git.sh
  verify_package.sh
  run_m3_owner.sh
```

15. 테스트 정보

15.1 최소 계층

- 26. unit: 순수 계산 · 상태기계 · JCS digest
- 27. contract: schema · 필드 · exit code · CLI
- 28. actual integration: 실제 제품 entrypoint와 승인 test fixture
- 29. mutation: 필드 · 파일 · 순서 · 정책을 의도적으로 파괴
- 30. adversarial: serialization, symlink, swap, oversized event, partial tail, path leak
- 31. target preflight: M3 · Metal · egress · memory source 확인

15.2 필수 mutation 범위

패키지의 `attack_matrix_v2.8.csv`에 정의한 58건을 모두 자동화한다. 각 테스트는 원래 artifact를 복제한 임시 공간에서 하나의 변이만 적용하고, 예상 exit code와 fail_class를 검증한다. 변이 문자열 자체가 배포 evidence 로그에 노출되지 않도록 scanner 결과는 finding ID와 digest만 출력한다.

15.3 테스트 진실성

- fake lambda와 SimpleNamespace만으로 실제 entrypoint PASS를 주장하지 않는다.
- mock은 OS 오류 주입과 단위테스트에만 사용한다.
- integration test는 production import 경로를 기록하고 coverage에서 핵심 gate 호출을 확인한다.
- mutation test가 하나라도 PASS를 깨지 못하면 verifier는 fail-open으로 간주한다.

16. CI 12-job과 의존관계

각 job은 signed source identity와 policy digest를 입력으로 받고, 이전 job의 boolean이 아니라 artifact digest를 검증한다.

순서	job	핵심 출력
1	source_provenance	full OID, tree, submodule/LFS, clean tree
2	policy_complete	UNSET 0, policy digest
3	compile_type_lint	toolchain lock, compile/type/lint
4	unit_contract	unit · schema · CLI
5	actual_integration	실제 entrypoint · coverage
6	mutation_adversarial	58 attack/mutation 전부 expected BLOCK
7	closed_world_schema	unknown/missing/orphan 차단
8	raw_path_secret_scan	전 로그 finding 0
9	nonactivation_route_diff	runtime off, 제품 의미 변화 0
10	package_from_git_object	archive · manifest · attestation · offline verify
11	swift_metal_typecheck	승인 Xcode/Swift에서 Metal · memory probe compile/typecheck
12	signed_attestation_replay	trust policy 서명 검증 · 독립 verifier 재생 · subject 일치

CI runner에서 실제 M3 · Metal · OS egress를 실행하지 못하면 NOT_TARGET_EXECUTED로 표기한다. 이를 PASS로 바꾸지 않는다.

target M3 preflight가 별도 11종 게이트를 닫는다.

17. M3 실행 런북

17.1 실행 전

32. 승인 commit과 패키지를 M3 Max에 가져온다.
33. 네트워크 의존 다운로드를 모두 끝내고 오프라인 모드로 전환한다.
34. 전체 OID, model manifest, runtime config, policy, fixture manifest를 검증한다.
35. OS egress controller를 활성화하고 native/subprocess probe가 차단되는지 확인한다.
36. thermal, Low Power Mode, power source, RAM, macOS/Xcode/Metal 정보를 수집한다.
37. 제품 preflight와 offline verifier self-test를 실행한다.

17.2 M3 시작 게이트 11종

```
P0_CODE_CLOSED=YES
ALL_46_DEFECTS_CLOSED=YES
CLEAN_COMMITTED_TREE=YES
GIT_BLOB_MATCH=YES
CI_GREEN=YES
OS_EGRESS_RECEIPT_VALID=YES
PRODUCT_PREFLIGHT_PASS=YES
INDEPENDENT_PREFLIGHT_VERIFY=YES
TRUST_POLICY_CONFIGURED=YES
```

위 목록과 별도로 BASE_FULL_SHA_RESOLVED=YES, BENCHMARK_POLICY_COMPLETE=YES도 필수다. 하나라도 NO이면 owner orchestrator는 worker를 생성하기 전에 종료한다. 외부 shell이 만든 all-true JSON은 입력으로 받지 않으며 각 gate의 signed receipt digest를 내부 검증한다.

17.3 단일 명령

```
./scripts/run_m3_owner.sh \
--policy benchmark_policy.json \
--fixture-manifest fixtures/manifest.jcs.json \
--candidate-a approved/model_1_7b.manifest.jcs.json \
--candidate-b approved/model_4b.manifest.jcs.json \
--output evidence/run-<RUN_ID>
```

CLI는 사전 게이트를 내부에서 다시 확인한다. --skip-*, --force-pass, test asset fallback, network fallback 옵션을 제공하지 않는다.

17.4 종료 후

38. worker와 자손이 모두 종료됐는지 확인한다.
39. artifact verifier를 별도 환경에서 실행한다.
40. raw/path/secret scan을 전체 디렉터리에 다시 실행한다.
41. final_verdict.json, verdict.normalized.json, SHA256SUMS, in-toto subject attestation을 생성한다.
42. 원시 민감 evidence는 정책에 따라 M3 안에서 삭제 또는 암호화 보관하고, 배포 ZIP에는 포함하지 않는다.

18. 역할과 거버넌스

주체	책임	할 수 없는 주장
알고리즘개발팀	품질 지표, evaluator, fixture·모델 정책, repair 알고리즘	M3 실행 없이 우수 모델 확정
메인개발팀	실제 제품 경로, terminal/DLP/grounding 통합, SDK/API 호환	fake 경로로 integration 완료
코덱스	하네스·CLI·스키마·패키징·검증기·테스트 구현	코드만으로 M3 성능 PASS
문제점검팀/보완팀	결함 추적, fail-open 감사, 지시서 정합성	원본 없이 실행 사실 추정
재검토팀	독립 코드·증거 재검토	producer boolean 재인용
대표/M3 담당	승인 M3에서 canonical CLI 실행	게이트 우회·수동 증거 수정
그룹A	M3 통과 후 M4 통합·shadow eval	M3만으로 G1_READY 선언

거버넌스 순서:

```
코드 구현 → 문제점검/보완 재감사 → 재검토 → 대표 승인·머지
→ M3 target preflight → 대표 M3 실측 → offline 독립검증
→ 총괄 raw 확인 → 통합앱 → 그룹A M4 → 별도 G1 판정
```

19. 산출물과 파일 구조

```
release/
  README.md
source/
  product-source.tar.gz
  harness-source.tar.gz
  product-git-tree.txt
  harness-git-tree.txt
  source-provenance.intoto.json
policy/
  benchmark_policy.json
  fixture_manifest.jcs.json
  model_A.manifest.jcs.json
  model_B.manifest.jcs.json
schemas/
ci/
evidence/
  run-<RUN_ID>/
    receipts/
    metrics/
    verifier/
    safe_logs/
    artifact_manifest.jcs.json
    final_verdict.json
    verdict.normalized.json
tests/
  mutation_results.json
SHA256SUMS
```

배포 ZIP에는 원문 prompt/output/reference, 개인 홈 경로, 모델 절대경로, secret, 암호화되지 않은 private evidence를 포함하지 않는다.

20. 최종 승인 기준

20.1 코드 병합 기준

- acceptance matrix의 P0·P1 및 결함 46건 연결 항목 100% PASS
- 58 attack/mutation 100% expected BLOCK
- actual integration과 route-diff PASS
- full OID·tree·Git blob·배포 SHA-256 검증 PASS
- CI 12-job PASS 또는 target-only 항목이 명시적으로 분리됨
- package offline verifier exit 0

20.2 M3 증거 기준

- 11개 시작 게이트 PASS
- schedule 예정 수와 실제 valid receipt 수가 정확히 일치
- invalid/aborted run 0, 또는 전체 새 run_id 재실행
- OS egress allowed attempt 0
- raw/path/secret finding 0
- dual resident 실제 overlap·양쪽 inference·메모리 PASS
- 사전 등록 품질·지연·메모리 정책에 따른 판정
- live semantic verifier와 offline artifact verifier 모두 PASS

20.3 완료 문구

실제 조건이 충족된 경우에만 다음 문구를 사용한다.

```
STATUS=M3_EVIDENCE_VALID_M4_PENDING
SPEC_REVIEW_PASS=YES
CODE_IMPLEMENTATION_PASS=YES
BASE_FULL_SHA_RESOLVED=YES
BENCHMARK_POLICY_COMPLETE=YES
TRUST_POLICY_CONFIGURED=YES
ALL_46_DEFECTS_CLOSED=YES
M3_START_ALLOWED=YES
M3_EVIDENCE_PASS=YES
LIVE_SEMANTIC_VERIFY_PASS=YES
OFFLINE_ARTIFACT_VERIFY_PASS=YES
G1_READY=NO
RUNTIME_ACTIVATION_ALLOWED=0
```

21. 금지 사항

- 짧은 SHA를 최종 불변 식별자로 사용
- 사후 가림 로그를 raw-zero PASS로 제출
- 원시 로그 byte 동일성을 재현성 기준으로 사용
- producer boolean, marker string, 파일 개수만으로 PASS
- 모델 디렉터리·샤드·tokenizer를 제외한 단일 파일 hash
- RSS > 0만으로 메모리 적합성 결론
- mock/fake lambda 결과를 실제 제품 integration으로 보고
- 정책값이 UNSET인데 임의 기본값으로 실행
- invalid epoch 일부만 골라 제거
- M3 없이 1.7B/4B 성능·품질·적합성 결론
- M4 없이 G1_READY=YES

22. 구현 우선순위

단계	구현	종료 조건
1	full OID · policy · schema 정본	짧은 SHA · UNSET 차단 테스트 PASS
2	VolatileDraft · terminal gate	serialization · DLP override mutation PASS
3	model manifest · worker FSM	swap · symlink · partial tail mutation PASS
4	memory/Metal · dual resident	target probe와 same-file mutation PASS
5	live/offline verifier	forged boolean · orphan · schedule mutation PASS
6	actual integration · CI 12-job	production call graph · route-diff PASS
7	git-object package	offline extraction · digest verify PASS
8	대표 M3 실측	M3 evidence + 양 verifier PASS
9	그룹A M4	별도 통합 기준 통과

23. 2026년 기준 근거

이 지시서의 외부 기술 기준은 2026-07-14에 다음 공식 자료를 확인해 반영했다.

43. MLCommons, MLPerf Client v1.6 (2026-04-06): Apple 플랫폼에서 MLX/Metal 및 llama.cpp/Metal을 포함한 client AI 벤치마크 최신 릴리스. <https://mlcommons.org/2026/04/mlperf-client-v1-6/>
44. SLSA Specification v1.2 Source Track: 전체 source revision, 변경 이력, provenance 및 기술 통제. <https://slsa.dev/spec/v1.2/source-requirements>
45. in-toto Attestation Framework, Statement v1: subject digest와 predicate 결속. <https://in-toto.io/Statement/v1>
46. RFC 8785 JSON Canonicalization Scheme: 불변 JSON bytes와 digest 계산. <https://www.rfc-editor.org/rfc/rfc8785.html>
47. JSON Schema Draft 2020-12: closed-world schema 및 unevaluatedProperties. <https://json-schema.org/draft/2020-12/json-schema-core>
48. Apple Foundation ProcessInfo.thermalState, isLowPowerModeEnabled: macOS 열·전원 상태.
<https://developer.apple.com/documentation/foundation/processinfo/thermalstate-swift.property> and
<https://developer.apple.com/documentation/foundation/processinfo/islowpowermodeenabled>
49. Apple Metal Device Inspection: current allocated size와 recommended maximum working set.
<https://developer.apple.com/documentation/metal/device-inspection>
50. Git git archive 공식 문서: commit/tree 입력과 archive metadata, export attributes.
<https://git-scm.com/docs/git-archive>
51. OWASP Logging Cheat Sheet: 로그에서 제외해야 할 민감정보 통제.
https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html

부록 A. 보완팀5 종결 판정

v2.8 지시서는 v2.7의 32개 결함을 유지·추적하면서 10개 상위 계약 결함을 추가 교정했다. 그러나 첨부물에 실제 저장소와 전체 SHA, 코드, CI, M3 raw가 없으므로 이 문서는 코드·성능 완료 확인서가 아니다. 알고리즘개발팀과 메인개발팀은 이 정본의 acceptance matrix와 mutation matrix를 코드로 구현하고, 실제 출력으로만 다음 상태를 올린다.

```
CURRENT=SPEC_READY
NEXT=CODE_READY_FOR_AUDIT
BLOCKERS=BASE_FULL_SHA_RESOLUTION,BENCHMARK_POLICY_VALUES,CODE_AND_EVIDENCE
```